

Kubernetes Complete Notes

1. What is Virtualization?

Virtualization is a technology that allows you to create multiple simulated environments or virtual machines (VMs) on a single physical hardware system.

- It enables **resource sharing**—CPU, memory, and storage—among multiple systems.
- Each VM runs its own **operating system** (OS) and acts as an independent computer.

Types of Virtualization

- **Server Virtualization:** Multiple servers run on a single physical machine.
- **Desktop Virtualization:** Multiple desktop environments hosted on a central server.
- **Network Virtualization:** Combines multiple network resources into one.
- **Storage Virtualization:** Pools storage devices for centralized management.

Hypervisors

Software used to create and manage VMs:

- **Type 1 (Bare Metal):** Runs directly on hardware (e.g., VMware ESXi, KVM, Hyper-V)
 - **Type 2 (Hosted):** Runs on top of an OS (e.g., VirtualBox, VMware Workstation)
-

2. Difference Between Virtual Machines and Containers

Feature	Virtual Machine (VM)	Container
Isolation	Full hardware & OS isolation	OS-level process isolation
Boot Time	Minutes	Seconds
Size	Heavy (GBs)	Lightweight (MBs)
Performance	Slower due to full OS	Fast and efficient

Feature	Virtual Machine (VM)	Container
	overhead	
Deployment	Complex	Simple & Portable
Example Tools	VMware, VirtualBox	Docker, Podman

→**In short:** VMs virtualize hardware; containers virtualize the OS.

3. Containerization and Docker

What is Containerization?

Containerization is a lightweight form of virtualization that packages an application and its dependencies into a **container**. This ensures the app runs consistently across any environment.

What is Docker?

Docker is the most popular containerization platform that automates:

- Building (via Dockerfile)
- Packaging (via Images)
- Running (via Containers)

Key Docker Concepts

- **Image:** Blueprint of the application.
 - **Container:** Running instance of an image.
 - **Dockerfile:** Set of instructions to build an image.
 - **Docker Hub:** Repository for sharing images.
-

4. Introduction to Kubernetes (K8s)

Kubernetes (K8s) is an open-source platform designed to automate:

- Deployment
- Scaling
- Management of containerized applications

It was developed by Google and is now maintained by the CNCF (Cloud Native Computing Foundation).

Why Learn Kubernetes?

- Handles container **orchestration** automatically.
- Ensures **high availability** and **scalability**.
- Enables **self-healing** (restarts failed containers automatically).
- Provides **load balancing** and **auto-scaling**.
- Used widely in **DevOps, Cloud, and Microservices** environments.

Real-World Value

- Increases deployment speed.
 - Reduces downtime.
 - Makes your applications cloud-ready.
-

5. Kubernetes Architecture & Building Blocks

Core Components

Kubernetes has two main parts: **1. Control Plane** and **2. Worker Nodes**

Control Plane Components

1. **API Server:** Entry point for all commands (kubectl communicates here).
 2. **etcd:** Key-value store that holds the cluster's configuration data.
 3. **Controller Manager:** Ensures the cluster state matches the desired state.
 4. **Scheduler:** Decides which node runs a new pod.
 5. **Cloud Controller Manager:** Integrates Kubernetes with cloud services.
-

Worker Node Components

1. **Kubelet:** Communicates with the control plane and manages containers.
 2. **Kube-proxy:** Handles network communication.
 3. **Container Runtime:** Runs containers (e.g., Docker, containerd).
-

Key Building Blocks

Component	Description
Cluster	A collection of nodes (worker +

Component	Description
	master)
Node	A worker machine (VM or physical)
Pod	The smallest deployable unit (holds one or more containers)
ReplicaSet	Ensures specified number of pod replicas are running
Deployment	Manages ReplicaSets and rolling updates
Service	Exposes pods to the network
Namespace	Logical partition in the cluster for resource isolation

6. Networking in Kubernetes

Kubernetes networking ensures all pods can communicate with each other, regardless of node.

Service Types

1. **ClusterIP (default):** Internal communication only within cluster.
2. **NodePort:** Exposes application on a static port of each node (e.g., :30000-32767).
3. **LoadBalancer:** Integrates with external cloud load balancers.
4. **ExternalName:** Maps service to external DNS.

Port Forwarding

Used to access a pod directly for testing/debugging:

```
kubectl port-forward pod/myapp 8080:80
```

This maps local port 8080 to pod port 80.

7. Storage & Volumes in Kubernetes

Containers are **ephemeral**, meaning their data is lost when restarted. To persist data, Kubernetes uses **Volumes**.

Types of Volumes

- **emptyDir**: Temporary storage (deleted when pod stops)
 - **hostPath**: Mounts host directory
 - **PersistentVolume (PV)**: Cluster-wide storage resource
 - **PersistentVolumeClaim (PVC)**: Request for storage by a pod
 - **ConfigMap/Secret**: Store configuration or sensitive data
-

8. Labels, Selectors, and Taints

Labels

- Key-value pairs attached to objects like pods.
- Used to group and select resources. Example:

```
labels:  
  app: frontend
```

Selectors

Used by services or deployments to match pods with specific labels.

```
selector:  
  matchLabels:  
    app: frontend
```

Taints and Tolerations

- **Taints**: Restrict pods from being scheduled on certain nodes.
- **Tolerations**: Allow specific pods to run on tainted nodes.

Example:

```
kubectl taint nodes node1 key=value:NoSchedule
```

9. Application Upgrades and Rolling Updates

Rolling Updates

- Replace old pods gradually with new pods.
- Ensures **zero downtime** during deployment.

Example:

```
kubectl rollout status deployment/myapp  
kubectl rollout undo deployment/myapp
```

Strategies

- **RollingUpdate:** Default; updates incrementally.
 - **Recreate:** Deletes all old pods first, then deploys new ones.
-

10. Hands-on Workflow Example

1. Create a Deployment

```
kubectl create deployment myapp --image=nginx
```

2. Expose the Deployment

```
kubectl expose deployment myapp --type=NodePort --port=80
```

3. Check Pods and Services

```
kubectl get pods  
kubectl get svc
```

4. Access Application Visit:

```
http://<NodeIP>:<NodePort>
```

5. Scale Application

```
kubectl scale deployment myapp --replicas=3
```

6. Upgrade Application

```
kubectl set image deployment/myapp nginx=nginx:latest
```

Summary

Concept	Description
Virtualization	Simulates hardware; runs multiple OSes
Containerization	Lightweight OS-level virtualization
Docker	Platform for building & running containers
Kubernetes	Orchestrator for managing containers
Pod	Basic deployable unit
Service	Exposes pods
Deployment	Manages rolling updates
Volume	Persists data
Label/Selector	Organizes and filters resources
Taint/Toleration	Controls pod scheduling